

Data Profiling for Change Rules

Nishttha Sharma^[0009–0009–0729–0761] and Fei Chiang^[0000–0003–4128–8074]

Dept. of Computing and Software
McMaster University, Hamilton, ON, Canada
{sharmn99, fchiang}@mcmaster.ca

Abstract. Understanding data change is critical towards understanding trends, normal vs. abnormal behaviours, recognizing patterns, and the causes of change. Existing database systems have limited support for change management, relying on statistics, triggers, and constraints. Data quality rules model sequential changes along a restricted set of attributes, quantify change among unordered tuples, and have limited ability to model the context under which attribute changes occur. In this paper, we introduce *Change Rules (CRs)* that quantify the sequential changes among ordered tuples in both the antecedent and consequent attributes. CRs aim to address the limitations of existing declarative dependencies to support trend analysis and causal relationships that trigger change among attributes. We propose CR-Miner, an automated algorithm for CR discovery that generates candidate change intervals in a level-wise manner. Experimental results show that CR-Miner achieves an average runtime improvement of 40–50% over existing baselines.

Keywords: change exploration · data profiling · change rules

1 Introduction

Data analysts seek to understand the cause and impact of change: What changes occurred (in the last day)? What caused these changes? Do similar update patterns exist? Was there an event that triggered a large set of changes, and what was the impact? How long have these changes been occurring? How can we minimize the impact of an undesirable change? Organizations have little intuition and knowledge of the types of data changes occurring in their systems to answer these questions. Existing database technology provides extensive features to efficiently retrieve and analyze data. However, database functionality to support change management is limited to vendor products that mine transaction logs, triggers, and check constraints that react to specific value changes, statistics that are re-generated for query optimization, and explanations for query execution plans. There is minimal work towards exploring, identifying, and analyzing change in data, and its impact on data quality. This lack of tools for interpreting temporal and spatial changes hinders our ability to completely understand the evolution of an entity, its evolving relationships to other entities, and trends.

Real-world data continuously changes over time. This evolution introduces challenges in understanding and managing changes effectively. These changes often carry critical information, revealing patterns and trends that are essential

Table 1: Patient lab results

	RBC	Hbg	Hct	WBC	Platelets
r_1	3.02	9.7	28.2	6.2	80
r_2	3.2	10.1	28.6	11.3	108
r_3	3.2	10.1	28.8	13.4	107
r_4	3.52	10.7	31.8	8.7	108
r_5	3.72	11.4	29.4	10.9	103
r_6	3.82	11.8	30.9	13.4	129
r_7	4.01	12	36.2	11.2	123
r_8	4.13	12.9	38	12.1	141

Table 2: Dependencies for Table 1.

Type	Dependency
OD	$\text{RBC} \mapsto \text{Hbg}$
SD	$\text{RBC} \rightarrow_{(0,1)} \text{Hbg}$
DD	$\text{RBC}(\leq 1.2) \rightarrow \text{Hbg}(\leq 3.5)$
CR	$\text{RBC}_{(0,2)} \rightarrow_{(0.2,0.9)} \text{Hbg}$

for understanding environmental conditions, system, and user behaviour. From a data quality perspective, identifying unexpected changes is critical to recognizing malicious activity and improper data use. Declarative methods have proposed data quality rules such as Order Dependencies (ODs) that capture relative order among tuples according to a set of attributes (e.g., $\text{RBC} \mapsto \text{Hemoglobin}$), but do not quantify the magnitude of change [19]. Sequential Dependencies (SDs) constrain the amount of change in a consequent attribute across ordered tuples (along the antecedent attribute) but do not quantify the change in the antecedent attributes [6]. Differential Dependencies (DDs) capture bounded differences between tuple pairs along a set of attributes, but are defined over *unordered* tuples, and do not model changes between ordered tuples [18]. Consider the following example highlighting limitations of existing data quality rules.

Example 1. Table 1 shows a sample of real patient blood lab results collected at various time points from the MIMIC-III [8]. The records represent Red Blood Cell (RBC), Hemoglobin (Hbg), Hematocrit (Hct), White Blood Cell (WBC), and Platelet counts. Table 2 shows defined OD, SD and DD rules over this data. The OD states that for tuples ordered (in ascending order) on RBC, the values in Hbg should also be ascending. There are no specifications on the consecutive differences between tuple values in the antecedent nor consequent attributes. The SD states that for tuples ordered on RBC, sequential changes in Hbg should occur in the range of $(0, 1)$, but does not define the ranges in which antecedent attribute changes (RBC) should occur. Lastly, the DD states that if the pairwise difference among all tuple pairs in RBC is less than 1.2, then the difference between the same tuple pairs in Hbg should be less than 3.5. However, these bounds are defined for non-ordered tuple pairs, making them inapplicable to identify sequential changes and trends *in both antecedent and consequent attributes*.

We address these limitations by introducing a new type of rule called *Change Rules (CRs)* that define bounded changes in both the antecedent and consequent attributes. For example, a CR such as the one shown in Table 2, specifies that if the difference between consecutive RBC attribute values occurs between $(0, 2)$, then the corresponding difference in Hbg should be in the range $(0.2, 0.9)$.

However, not all observed changes necessarily contribute equally when determining these bounds. For example, the highlighted tuples (r_4, r_5) show a small

increase in RBC and Hbg but a decline in Hct. Such transitions introduce irregular variation in Hct despite stable trends in closely related attributes. To determine whether a data change is normal (or not), it is necessary to consider related attributes that provide context of the change. We study how to compute a scaling factor that adjusts data changes for this necessary context.

Challenges. Our work aims to identify relevant changes that are contextualized with related attributes, and to develop an algorithm for change rule discovery. We address the following challenges:

- (1) Identifying relevant context for attribute changes: changes among attribute values are not always proportional and occur within a context of related attributes. For example, a larger salary increase in a given year may be due to an increase in the number of employees, lower spending costs, and increased sales revenue. Identifying relevant attributes and their changes, particularly with a large number of attributes and records, requires efficient, scalable solutions.
- (2) Manual definition of CRs is time-consuming and requires domain expertise. Mining CRs is challenging for growing schema and data size, as we must consider the scalability of rule discovery to provide effective pruning strategies.
- (3) Given the large number of possible CRs, identifying meaningful and relevant CRs requires a notion of *minimality*. Intuitively, minimal CRs should have the fewest number of antecedent and consequent attributes, and the shortest interval for each attribute, while satisfying minimum frequency (support) thresholds.

Contributions. We make the following contributions:

- (1) Introduce and formally define *Change Rules (CRs)* that extend existing data dependencies to consider bounded changes in the antecedent and consequent attributes among an ordered set of tuples.
- (2) CR-Miner: a new sequential algorithm for CR discovery. CR-Miner efficiently generates differences among tuples (called differential functions), and uses these functions to compute minimal width intervals that maximize the number of satisfying pairwise tuples, satisfying a predefined support threshold. We leverage bitset representations for support computation via longest consecutive subsequences for improved algorithmic efficiency.
- (3) We propose a methodology to contextualize changes that adjust changes in a target attribute using a learned context factor β . This contextualization provides explainability for ‘out-of-range’ changes that may appear abnormal, but with appropriate context, improves explainability and ease of interpretation.
- (4) Experimental system evaluation. We evaluate CR-Miner on three real-world datasets, MIMIC-III, Employment, and Weather, to validate its scalability and effectiveness. We show that CR-Miner significantly outperforms existing baselines, achieving a 40-80% speedup over FastDD on the MIMIC-III and Employment datasets, and a 50-85% speedup on the Weather dataset.

2 Preliminaries

We define the necessary definitions and notation.

Data model. Consider a relational schema $R = \{A_1, A_2, \dots, A_N\}$, and an instance $I = \langle r_1, r_2, \dots, r_m \rangle$, where each tuple r_k assigns a value $r_k[A_i]$ to attribute $A_i \in R$. We impose an ordering on I based on attribute values. For a given attribute $A_i \in R$, the tuples are sorted in ascending order of A_i , producing an ordered sequence $\pi_i(I) = \langle r_1, r_2, \dots, r_m \rangle$ such that $r_1[A_i] \leq r_2[A_i] \leq \dots \leq r_m[A_i]$. This ordering captures how other attributes evolve as the value of A_i increases.

Data change. Given an ordered sequence $\pi_i(I) = \langle r_1, r_2, \dots, r_m \rangle$, we define the change in $A_j \in R$ between two consecutive tuples r_k and r_{k+1} as:

$$\delta_{(k,k+1)}[A_j] = d(r_k[A_j], r_{k+1}[A_j])$$

where $d(\cdot)$ is a distance function that quantifies the change in A_j between the two tuples. Each consecutive tuple pair (r_k, r_{k+1}) is represented as a **change vector** as follows: $\vec{r}_k = (\delta_{(k,k+1)}[A_1], \delta_{(k,k+1)}[A_2], \dots, \delta_{(k,k+1)}[A_N])$.

Differential functions. Differential functions impose constraints on the difference between attribute values across tuple pairs [18]. A differential function $\lambda(A_j)$ over an attribute A_j is of the form: $\lambda(A_j) = (\delta_{(k,k+1)}[A_j] \in [l, u])$, where l and u denote the lower and upper bounds on the change in A_j . The function $\lambda(A_j)$ is satisfied for a tuple pair (r_k, r_{k+1}) if the change in A_j lies within the interval. Let Ψ denote the set of all differential functions across all attributes.

Diff-sets. Diff-sets were introduced to efficiently encode violations of differential functions [10]. The diff-set for a tuple pair is the set of all differential functions in Ψ that are not satisfied by the change $\delta_{(k,k+1)}[A_j]$ ($\forall A_j \in R$). Formally, the diff-set of a tuple pair is $D(r_k, r_{k+1}) = \{\lambda(A_j) \in \Psi \mid \delta_{(k,k+1)}[A_j] \notin [l, u]\}$. The set of all such diff-sets is denoted as D . For a specific function $\lambda(A_j)$, we define $D(\lambda(A_j)) = \{U \in D \mid \lambda(A_j) \in U\}$, which represents the set of all tuple pairs that violate $\lambda(A_j)$.

3 Framework Overview

Figure 1 gives an overview of the framework, consisting of three phases: Preprocessing, Context-Aware Change, and CR-Miner. In preprocessing, we order I by X , and compute consecutive changes for each attribute $A_j \in R$ as $\delta_{(k,k+1)}[A_j] = d(r_k[A_j], r_{k+1}[A_j])$.

Context-aware changes. For a given target attribute A_t , we seek to identify changes that lie outside of an expected range. We do this by defining a window of size $\mathcal{W}$ and computing the z-score of each $\delta_{(k,k+1)}[A_j]$, $\forall A_j \in R$ producing a change vector $\vec{z}_k = (z_k[A_1], z_k[A_2], \dots, z_k[A_N])$. Using statistical z-score cut-offs, we compute normal vs. abnormal labels with respect to the window $\mathcal{W}$ of changes. These vectors are the input to a Random Forest classifier that is used to identify whether changes in A_t are normal or not, and the relative feature weights among the attributes $A_j \in R \setminus A_t$. We use these weights to compute a context factor β for each tuple pair of changes that adjusts the changes in A_t .

CR-Miner. For each attribute $A_j \in R$, we define the set of sorted changes as S_{A_j} . Candidate intervals are generated from S_{A_j} by computing a bitset for each inter-

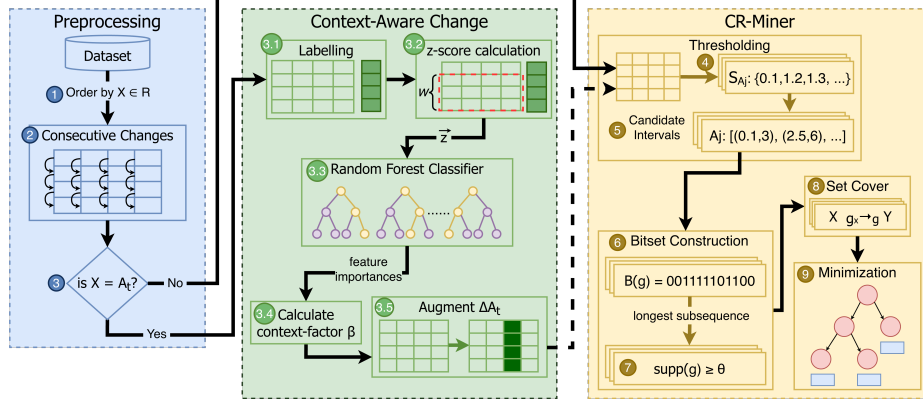


Fig. 1: Framework Overview

val where the bit is set to 1 if the corresponding change lies in the given interval, and 0 otherwise. If the longest consecutive sequence of 1's for an interval gap g satisfies the minimum segment coverage θ_c , g is retained as a valid differential function. Using these differential functions, we generate candidate CRs of the form $X \xrightarrow{g_x} Y$ via a set cover procedure over the corresponding bitsets of g_x and g whose overlap satisfies a minimum support threshold θ . We seek minimal, non-redundant rules where the antecedent and consequent intervals cannot be further narrowed without reducing the support of the rule.

4 Context-Aware Changes

Given an ordered sequence $\pi_t(I) = \langle r_1, r_2, \dots, r_m \rangle$, each tuple pair (r_k, r_{k+1}) is associated with a change vector $\vec{r}_k$. For each attribute $A_j \in R$, we construct a window of the previous $\mathcal{W}$ change values $\{\delta_{(k-w, k-w+1)}[A_j], \dots, \delta_{(k-1, k)}[A_j]\}$. From this window, we compute the mean $\mu_k[A_j]$ and standard deviation $\sigma_k[A_j]$, as shown in Figure 2. To make changes comparable across attributes with different scales, we apply z-normalization: $z_k[A_j] = \frac{\delta_{(k, k+1)}[A_j] - \mu_k[A_j]}{\sigma_k[A_j] + \epsilon}$, where ϵ is a small constant for numerical stability. Each tuple pair (r_k, r_{k+1}) is thus represented as a normalized change vector $\vec{z}_k = (z_k[A_1], z_k[A_2], \dots, z_k[A_N])$.

Learning weights. For a chosen target attribute A_t , we train a classifier using $\vec{z}_k$ as input and labels indicating whether the observed change $\delta_{(k, k+1)}[A_t]$ is normal or not. A change is marked abnormal if $z_k[A_t]$ exceeds a fixed threshold or significantly deviates from contextual behaviour. Formally, for each tuple pair, we compute a context baseline using the average z-score of contextual attributes, and assign a positive label if $z_k[A_t] > \eta$ or $z_k[A_t] > \nu \cdot \text{avg}(\text{context})$, where η is a fixed threshold and ν controls sensitivity to contextual deviation. The model learns a mapping $f : \vec{z}_k \rightarrow p_k$, where p_k represents the predicted contextual behaviour of the target change. From the trained model, we extract attribute importance weights $\{w_1, w_2, \dots, w_N\}$, where w_j reflects the contribution of attribute A_j to explain the changes in A_t .

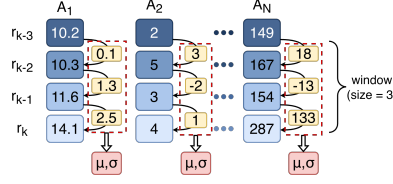
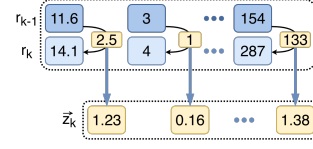
Fig. 2: Localized changes within window $\mathcal{W}$.

Fig. 3: Context adjustment.

Context factor. Using the learned weights, we define the context factor for tuple pair (r_k, r_{k+1}) as $\beta_k[A_t] = z_k[A_t] - \sum_{A_j \in R \setminus \{A_t\}} w_j \cdot z_k[A_j]$, measuring the deviation of the target attribute from expected changes in attributes A_j . We then scale the changes as: $\delta'_{(k,k+1)}[A_t] = \delta_{(k,k+1)}[A_t] \cdot \beta_k[A_t]$. We compute a sequence of context-aware change vectors, where A_t is adjusted based on high-weighted, contextual attributes and feed the resulting context-aware change vectors into the rule mining stage. The learned weights identify a subset of high-relevance attributes, $R_t \subseteq R \setminus \{A_t\}$, that most strongly influence changes in A_t . CR-Miner then utilizes R_t to effectively prune the search space.

Identifying the relevant attributes for A_t . We sort the weights for $R \setminus \{A_t\}$, in descending order ($w_{(1)} \geq w_{(2)} \geq \dots \geq w_{(N-1)}$) and compute the differences as $\Delta_i = w_{(i)} - w_{(i+1)}$, $1 \leq i < N-1$. The index of the largest drop is identified as $c^* = \arg \max_i \Delta_i$. This largest gap represents the point at which contextual relevance decreases most sharply. We therefore retain only the attributes preceding this gap as the relevant attribute set $\mathcal{R}_t = \{A_{(1)}, A_{(2)}, \dots, A_{(c^*)}\}$.

5 CR-Miner

We introduce our CR mining algorithm, CR-Miner, by first defining change rules and then how we build diff-sets to identify candidate intervals encapsulating attribute changes. We then use these intervals and incrementally enlarge them to identify non-redundant, minimal change rules.

Definition 1. (*Change Rules*) Let I be an instance ordered by an attribute $X \in R$, producing $\pi_X(I) = \langle r_1, \dots, r_n \rangle$. A change rule ϕ is defined as $\phi: X_{g_X} \rightarrow_g Y$, where $g_X = [l_X, u_X]$ and $g = [l, u]$ are intervals that capture the minimum and maximum range of changes in X and Y , respectively. That is, we say ϕ holds for a consecutive pair (denoted as $(r_k, r_{k+1}) \models \phi$), if $\delta_{(k,k+1)}[X] \in g_X \rightarrow \delta_{(k,k+1)}[Y] \in g$.

In this work, we assume X to be a singular attribute. Since CRs are mined from tuples ordered over X , the discovered rules are inherently conditioned on this ordering. Each X ordering is independent of the others. Different choices of X induce different changes leading to different change rules.

5.1 Building Diff-sets

A differential function on attribute A_j is defined as $\lambda(A_j) = (\delta_{(k,k+1)}[A_j] \in [l, u])$. To construct a set of differential functions Ψ , we must do so incrementally. For an attribute $A_j \in R$, let the sequence of changes between all tuple pairs of

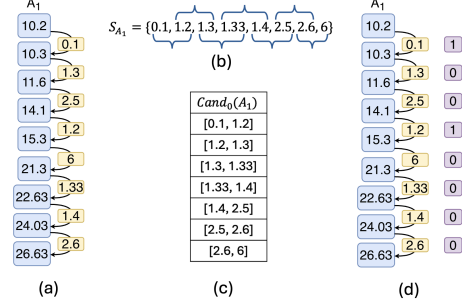


Fig. 4: Generating candidate intervals and their bitsets

$\pi_X(I)$ be $\Delta A_j = \{\delta_{(1,2)}[A_j], \dots, \delta_{(m-1,m)}[A_j]\}$, where each element in the set corresponds to a tuple pair (r_k, r_{k+1}) . Let $S_{A_j} = \{s_1, s_2, \dots\}$ be the sorted set of **distinct** change values in ΔA_j . For example, from Figure 4(a), $S_{A_1} = \{0.1, 1.2, 1.3, 1.33, 1.4, 2.5, 2.6, 6\}$.

We compute candidate intervals by starting with adjacent values in S_{A_j} , as shown in Figure 4(b). The first set of candidate intervals for A_1 is shown in Figure 4(c). We proceed to increase the size of the intervals by incrementally expanding the upper bound. Let the set of candidate intervals proceed in a levelwise manner starting at level 0, defined as $Cand_0(A_j) = \{[s_p, s_{p+1}] \mid s_p \in S_{A_j}, 0 \leq p \leq |S_{A_j}| - 1\}$. For each interval $g_i \in Cand_0(A_j)$, we iterate through ΔA_j to form a bitset $\mathcal{B}(g_i)$ of length $|\Delta A_j|$, where:

$$\mathcal{B}(g_i)[k] = \begin{cases} 1 & \text{if } \Delta A_j[k] \in g_i \\ 0 & \text{otherwise} \end{cases}$$

The k -th bit in the bitset represents if $\delta_{(k,k+1)}[A_j] \in g_i$ or not, as shown in Figure 4(d). We iteratively construct larger candidate intervals by combining adjacent intervals from the previous level. For level $\ell > 0$, candidate intervals are generated as: $Cand_\ell(A_j) = \{[s_p, s_{p+\ell+1}] \mid 0 \leq p \leq |S_{A_j}| - \ell - 2\}$. Instead of recomputing bitsets, we exploit the composability of adjacent intervals. For two consecutive intervals $g_a = [s_p, s_q]$ and $g_b = [s_q, s_r]$, the bitset of their merged intervals can be computed as $\mathcal{B}(g_a \cup g_b) = \mathcal{B}(g_a) \vee \mathcal{B}(g_b)$. For example, $\mathcal{B}([0.1, 1.3]) = \mathcal{B}([0.1, 1.2]) \vee \mathcal{B}([1.2, 1.3]) = 10010000 \vee 01010000 = 11010000$. For each candidate interval g_i , we compute the length of the longest consecutive subsequence of 1's in its bitset $\mathcal{B}(g_i)$. Let this length be denoted as $max_seg(g_i)$.

Definition 2. (*Segment Coverage*) The segment coverage of a candidate interval g_i is defined as the fraction of consecutive tuple pairs contained in its longest contiguous segment, i.e., $coverage(g_i) = \frac{max_seg(g_i)}{m-1}$.

To compute $max_seg(g_i)$ efficiently, we scan $\mathcal{B}(g_i)$ from left to right and only initiate counting at positions where the bit is 1. For each such position t , we extend forward to measure the length of the consecutive run of 1's starting at t . Let $\tau = \lceil \theta_c \cdot (m-1) \rceil$ be the minimum required segment length to satisfy the

segment coverage threshold θ_c . During the scan, if the current starting position is t , then even in the best case (i.e., all remaining bits are 1), the maximum possible segment length is $(m - 1 - t)$. Therefore, if $(m - 1 - t) < \tau$, we can terminate early, as no valid segment can be found beyond this point. If no segment of length at least τ is found, the candidate interval g_i is pruned. If a candidate interval g_i satisfies $\text{coverage}(g_i) \geq \theta_c$, it is added to the set of differential functions for attribute A_j , denoted as ψ_j .

5.2 Mining Change Rules

The set of differential functions across all attributes is denoted as $\Psi = \{\psi_{A_1}, \psi_{A_2}, \dots, \psi_{A_N}\}$, where $\psi_X \in \Psi$ contains the differential functions for the antecedent attribute X . Given an interval $g_X \in \psi_X$, CR-Miner identifies a differential function $g \in \psi_Y$ for some $Y \in R \setminus \{X\}$. When X is the target attribute A_t , the candidate consequent attribute is restricted to $\mathcal{R}_t$, the context-relevant attributes. Otherwise, the consequent is $R \setminus X$. A candidate change rule is discovered based on the co-occurrence of the longest valid segments of the differential functions of X and Y , provided that the resulting rule satisfies the minimum support threshold θ .

Definition 3. (*Support of a Change Rule*) The support of a change rule $\phi : X \xrightarrow{g_X} Y$ measures the fraction of the length of the longest consecutive segment of tuple pairs that simultaneously satisfy both the antecedent and consequent intervals. Let $\mathcal{I} = \{i \in \{1, \dots, |I| - 1\} \mid (r_i, r_{i+1}) \models \phi\}$ denote the set of positions whose consecutive tuple pairs satisfy the change rule ϕ . Then, $\max_{[a,b] \subseteq \mathcal{I}} (b - a + 1)$ will be the length of the longest consecutive segment of satisfying positions. The support of ϕ is defined as:

$$\text{support}(\phi) = \frac{\max_{[a,b] \subseteq \mathcal{I}} (b - a + 1)}{|I| - 1}$$

A CR ϕ is said to hold with threshold θ if $\text{support}(\phi) \geq \theta$.

5.3 Minimality of Change Rules

We define the minimality of a change rule based on two criteria: (i) the specificity of the change intervals, and (ii) the preservation of rule support. Let us consider two CRs over the same attribute pair: $\phi : X \xrightarrow{g_X} Y$, and $\phi' : X \xrightarrow{g'_X} Y$. An interval $g' = [l', u']$ is subsumed by $g = [l, u]$ (denoted $g' \preceq g$) if $[l', u'] \subseteq [l, u]$.

Intuitively, ϕ' is considered more minimal if it achieves a tighter characterization of change while maintaining nearly the same support as a broader rule. Therefore, a CR ϕ' is considered minimal if no other rule ϕ exists such that $g_X \preceq g'_X$, $g \preceq g'$, and $\text{support}(\phi) \geq \text{support}(\phi') - \epsilon_s$. Here, $\epsilon_s \geq 0$ is a support tolerance threshold, defined as a small fraction of the total number of tuples. That is, no rule with tighter antecedent or consequent intervals can achieve approximately the same support as ϕ' .

Algorithms. We now describe the details of CR-Miner, which is divided into two main algorithms: (i) BuildDiff, and (ii) Discover Change Rules.

Algorithm 1 BuildDiff**Input:** Ordered tuple sequence $\pi_X(I)$, segment coverage threshold θ_c **Output:** Differential functions Ψ and bitsets $\mathcal{B}$

```

1  $\Psi \leftarrow \emptyset, \mathcal{B} \leftarrow \emptyset$ 
2  $\tau \leftarrow \lceil \theta_c \cdot (m - 1) \rceil$ 
3 foreach  $A_j \in R$  do
4    $S_{A_j} \leftarrow$  sorted distinct values in  $\Delta A_j, \psi_{A_j} \leftarrow \emptyset$ 
5    $Cand_0 \leftarrow \{[s_p, s_{p+1}] \mid 0 \leq p \leq |S_{A_j}| - 2\}$ 
6   compute  $\mathcal{B}(g)$  for each  $g \in Cand_0$ 
7    $\ell \leftarrow 0$ 
8   while  $Cand_\ell \neq \emptyset$  do
9      $Cand_{\ell+1} \leftarrow \emptyset$ 
10    foreach  $g \in Cand_\ell$  do
11       $current\_run \leftarrow 0, max\_seg(g) \leftarrow 0, expandable(g) \leftarrow true$ 
12      for  $t = 1$  to  $m - 1$  do
13        break if  $(m - t) < \tau$ 
14        if  $\mathcal{B}(g)[t] == 1$  then
15           $current\_run \leftarrow current\_run + 1$ 
16           $max\_seg(g) \leftarrow \max(max\_seg(g), current\_run)$ 
17        else
18           $current\_run \leftarrow 0$ 
19        if  $\frac{max\_seg(g)}{m-1} \geq \theta_c$  then
20           $\psi_{A_j} \leftarrow \psi_{A_j} \cup \{g\}, \mathcal{B}[g] \leftarrow \mathcal{B}(g), expandable(g) \leftarrow false$ 
21      for  $i = 0$  to  $|Cand_\ell| - 2$  do
22         $g_a \leftarrow Cand_\ell[i], g_b \leftarrow Cand_\ell[i + 1]$ 
23        if  $expandable(g_a)$  and  $expandable(g_b)$  then
24           $g_{new} \leftarrow [l(g_a), u(g_b)]$ 
25           $\mathcal{B}(g_{new}) \leftarrow \mathcal{B}(g_a) \vee \mathcal{B}(g_b)$ 
26          add  $g_{new}$  to  $Cand_{\ell+1}$ 
27       $\ell \leftarrow \ell + 1$ 
28    $\Psi \leftarrow \Psi \cup \psi_{A_j}$ 
29 return  $\Psi, \mathcal{B}$ 

```

BuildDiff (Algorithm 1): constructs the set of differential functions Ψ and their bitset representations $\mathcal{B}$ using a level-wise candidate generation strategy. For each attribute $A_j \in R$, consecutive changes ΔA_j are computed from the ordered instance, and the distinct values are sorted to form S_{A_j} (Line 4). Level-0 candidates $Cand_0(A_j)$ are initialized using consecutive values in S_{A_j} , and their corresponding bitsets are constructed to indicate satisfying tuple pairs (Line 6). The algorithm proceeds in a level-wise manner (Line 8), where each candidate interval $g \in Cand_\ell$ is evaluated by computing the longest consecutive run of 1's in its bitset to derive segment coverage (Lines 9-18). Candidate intervals satisfying the segment coverage threshold θ_c are added to ψ_{A_j} , while intervals that fail to meet θ_c are marked non-expandable to prune the search space (Lines 19-20). An upward pruning strategy is applied using a minimum segment length threshold τ , allowing early termination when remaining positions cannot satisfy τ (Lines 20). Further candidates at subsequent levels are generated by merging adjacent intervals, and their bitsets are efficiently computed using bitwise OR

(Lines 21-27). The process continues until no further candidates can be generated, returning Ψ and $\mathcal{B}$.

Algorithm 2 Discover Change Rules

Input: Differential functions Ψ , bitsets $\mathcal{B}$, ordering attribute X , thresholds θ_c , θ , support tolerance ϵ_s , target attribute A_t , and set of corresponding relevance weights $\{w_j\}$ for each A_j

Output: Change Rule set Σ

```

1   $\Sigma \leftarrow \emptyset$ 
2   $\psi_X \leftarrow$  differential functions for attribute  $X$ 
3  if  $X = A_t$  then
4     $\mathcal{C} \leftarrow \mathcal{R}_t$ 
5  else
6     $\mathcal{C} \leftarrow R \setminus \{X\}$ 
7  foreach  $g_X \in \psi_X$  do
8     $b_X \leftarrow \mathcal{B}(g_X)$ 
9    foreach  $Y \in \mathcal{C}$  do
10      $\psi_Y \leftarrow$  differential functions for attribute  $Y$ 
11     foreach  $g \in \psi_Y$  do
12        $b_Y \leftarrow \mathcal{B}(g)$ 
13        $b_{AND} \leftarrow b_X \wedge b_Y$ 
14        $current\_run \leftarrow 0, max\_seg \leftarrow 0$ 
15       for  $t = 1$  to  $|I| - 1$  do
16         if  $b_{AND}[t] == 1$  then
17            $current\_run \leftarrow current\_run + 1$ 
18            $max\_seg \leftarrow \max(max\_seg, current\_run)$ 
19         else
20            $current\_run \leftarrow 0$ 
21       if  $support(\phi) \leftarrow \frac{max\_seg}{|I| - 1} \geq \theta$  then
22          $minimal \leftarrow \text{true}$ 
23         foreach  $\phi' \in \Sigma$  over same  $(X, Y)$  do
24           if  $g'_X \preceq g_X$  and  $g' \preceq g$  and  $support(\phi') \geq support(\phi) - \epsilon_s$  then
25              $minimal \leftarrow \text{false}$ 
26           break
27       if  $minimal$  then
28          $\Sigma \leftarrow \Sigma \cup \{\phi : X \xrightarrow{g_X} Y\}$ 
29 return  $\Sigma$ 

```

Discover Change Rules (Algorithm 2): identifies valid Change Rules Σ from the set of differential functions Ψ and their bitsets $\mathcal{B}$. The algorithm first extracts antecedent differential functions ψ_X for attribute X (Line 2). When $X = A_t$, the candidate consequent attribute set is restricted to $\mathcal{R}_t$. Otherwise, all attributes in $R \setminus \{X\}$ are considered (Lines 3–6). For each antecedent interval $g_X \in \psi_X$ and each consequent attribute $Y \in \mathcal{C}$, and each interval $g \in \psi_Y$, the algorithm computes the bitwise AND of $\mathcal{B}(g_X)$ and $\mathcal{B}(g)$ and scans the resulting bitset to find the longest consecutive run of 1's (Lines 12–20). This length is used to compute the support of the candidate rule ϕ as the fraction of the longest co-occurring consecutive segment over $|I| - 1$ (Line 21). If the support meets or exceeds the threshold θ , a minimality check is performed: the rule is added to

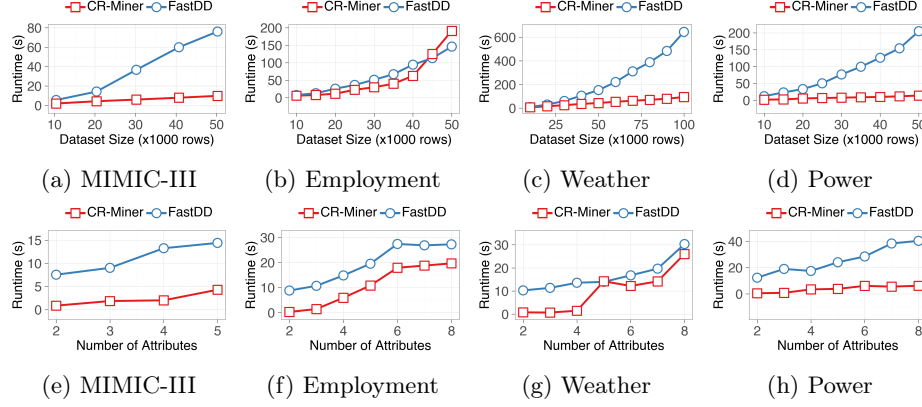


Fig. 5: Comparative runtime evaluation

Σ only if no existing rule over the same attribute pair has a tighter or equal antecedent and consequent intervals while achieving at least $support(\phi) - \epsilon_s$ (Lines 23–28). The process repeats for all interval pairs, returning the final rule set Σ .

6 Experiments

Setup Environment. We implement CR-Miner using Python 3.10.18, and conducted experiments in a local Conda environment, M1 chip, equipped with 8 GB RAM, with macOS 26.3.1. Unless otherwise stated, we used default values of $\theta = 0.9$, $\theta_c = 0.95$, $\epsilon_s = 0.01$, and $\mathcal{W} = 8$. For comparative experiments, we evaluate CR-Miner against FastDD, an existing DD discovery baseline algorithm [10]. We select FastDD as the baseline because it represents the closest available comparison to CR-Miner. They both model bounded differences between attributes and use diff-set-based mining over change intervals. CR-Miner extends the FastDD framework to ordered sequential data, whereas DDs are defined over all tuple pairs in the relation.

Datasets. We use three real-world datasets covering healthcare, employment, and weather domains. Our data and source code are publicly available [17].

(1) **MIMIC-III** [8]: describes the healthcare information of patients admitted to the emergency department and ICU at a hospital in Boston, MA, from 2001-2012. We focus on laboratory results covering 5 attributes and patient demographics over 50K records.

(2) **Employment** [20]: describes industry-wide employment distributions across the US states, describing attributes such as firm size, total payroll, and total number of employees. Our dataset contains 8 attributes and 50k records spanning employment statistics from 2012 to 2022.

(3) **Weather** [5]: this data is collected from the Environment and Climate Change Canada daily weather observations for the province of Ontario, spanning 2015

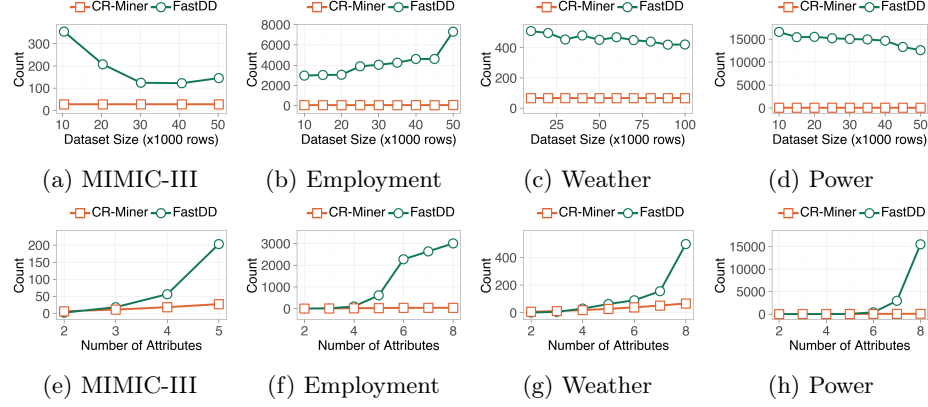


Fig. 6: Comparative number of discovered rules.

to 2025. The data includes provincial meteorological measurements such as temperature and precipitation. Our data includes 8 attributes and 100k records. Power [16]: this data describes the power consumption of three different distribution networks of Tetouan city, Morocco, in 2017. The data includes meteorological measurements and the power consumed in each zone every 10 minutes.

6.1 Experimental Results

Exp-1: Runtime vs. data size. Figure 5 (a)-(d) shows the runtimes of CR-Miner and FastDD as the dataset size increases across the three datasets. CR-Miner scales more efficiently on MIMIC-III, Weather, and Power, exhibiting near-linear growth with significantly lower runtime compared to FastDD. In contrast, FastDD shows a steeper increase, indicating higher sensitivity to dataset size due to pairwise comparisons. On the Employment dataset, CR-Miner initially outperforms FastDD but exhibits a sharper increase at larger sizes, eventually exceeding FastDD at larger data sizes. This suggests that runtime is also influenced by attribute characteristics and interval generation complexity.

Exp-2: Runtime vs. #attributes. Figure 5 (e)-(h) shows the runtime as the number of attributes increase (data size is fixed at 20K). Across all datasets, CR-Miner consistently maintains a lower runtime compared to FastDD as the number of attributes increases. For MIMIC-III and Employment, CR-Miner exhibits stable, near-linear scaling, whereas FastDD runtimes increase at a faster rate, reflecting the increased computational overhead of pairwise value comparisons for more attributes.

Exp-3: #Discovered rules vs. data size. Figure 6 (a)-(d) shows the number of discovered CRs using CR-Miner, and the number of discovered DDs using FastDD as the dataset size increases. As expected, FastDD identifies a larger number of rules given that DDs subsume CRs, as the differential functions are expected to hold over all pairwise tuples in I . In contrast, CRs hold over an ordered set of tuples. The latter case is particularly important when sequential

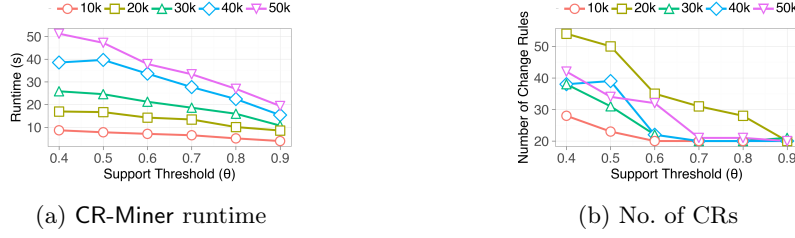


Fig. 7: Varying support threshold.

changes in both antecedent and consequent attributes are needed (among consecutive tuples) to identify causal relationships that are difficult to identify in DDs. CR-Miner shows a stable set of frequent CRs for varying data sizes.

Exp-4: #Discovered rules vs. #attributes. Figure 6 (e)-(h) shows the number of discovered CRs and DDs as the number of attributes increases (data size is fixed at 20K). As expected, FastDD shows an increased number of discovered rules across all datasets, due to an increased number of differential functions that must be evaluated, leading to more DDs. We note that many of these DDs contain partial overlap among attributes and among the differential functions. CR-Miner identifies non-redundant, minimal rules, leading to a more succinct and relevant set of rules.

Exp-5: Varying support. Figure 7a shows the CR-Miner runtime for varying support, and data sizes using the MIMIC-III dataset. For all data sizes, the runtimes linearly decrease as support levels increase, as expected. Higher support thresholds enable more aggressive pruning of the search space, as fewer CR candidates satisfy the stricter criteria. Consequently, Figure 7b shows the number of discovered CRs for varying support. As the support threshold increases, the number of discovered CRs decreases, as expected.

Exp-6: Example Change Rules. We present three examples of CRs mined from the Weather and MIMIC data.

ϕ_1 : [MinTemp] $_{(0,0.2)} \rightarrow_{(0,0.1)}$ [CoolDegDays]. MinTemp captures the lowest daily temperature, which typically occurs at night or before sunrise. CoolDegDays measures the energy to cool a building using air conditioning. Small or negligible changes in minimum temperature do not contribute to higher daytime temperatures, and therefore do not increase cooling demand. In comparison, the DD $\{[\text{MinTemp}(> 12.6)] \wedge [\text{MeanTemp}(\leq 0)] \rightarrow [\text{CoolDegDays}(> 1.3)]\}$ does not model how changes in MinTemp propagate sequentially to changes in cooling demand. Due to the framework of DDs, it cannot distinguish whether MinTemp is rising or falling, nor whether the change in CoolDegDays is an increase or a decrease. ϕ_1 provides a more interpretable view of how incremental temperature changes translate into bounded changes.

ϕ_2 : [Hct] $_{(0,0.2)} \rightarrow_{(-2.7,2.7)}$ [Hbg]. ϕ_2 reflects the well-established relationship between Hematocrit (Hct) and Hemoglobin (Hbg), commonly approximated by the clinical "The Rule of Three," where $\text{Hct} \approx 3 \times \text{Hbg}$ [3]. The CR ϕ_2 exem-



Fig. 8: Studying the effects of changing window size.

plifies this constraint, showing a restricted change of Hbg up to 3x; reflecting a clinical biological constraint. In comparison, the DD $\{[\text{Hct}(> 29.7)] \wedge [\text{RBC}(\leq 0)] \rightarrow [\text{Hbg}(> 10.1)]\}$ identifies attribute differences that co-occur. Since DDs use absolute change, discarding direction, a decrease in Hct is indistinguishable from an increase. Consequently, a clinically anomalous pattern where Hbg rises significantly while Hct falls is undetectable by this DD, whereas ϕ_2 can capture this anomaly, making it more actionable for sequential patient data analysis.

ϕ_3 : $[\text{Humidity}]_{(-6.7, 96.8)} \rightarrow (-9211.14, 5711.34) [\text{Zone1}]$. ϕ_3 captures a strong environmental - to - load dependency, where variations in humidity correspond to substantial changes in zone energy demand. This reflects the sensitivity of HVAC-driven systems to atmospheric conditions, where humidity variations influence cooling and dehumidification requirements, thereby affecting overall power consumption. In comparison, the DD $\{[\text{Temperature}(\leq 0.2)] \wedge [\text{Humidity}(\leq 0.0)] \wedge [\text{gendiffuseflows}(\leq 0.01)] \wedge [\text{Zone3}(\leq 0.0)] \rightarrow [\text{Zone1}(> 6000.0)]\}$ only captures a conjunction of conditions leading to high Zone 1 consumption, without modeling how changes evolve across consecutive observations. Furthermore, the inclusion of Zone 3 power consumption in the antecedent primarily reflects co-occurrence rather than a change relationship.

Exp-7: Varying window size $\mathcal{W}$. Figure 8 (a)-(c) shows the impact of varying the window size across different dataset sizes (10k to 50k) using the MIMIC-III dataset. As shown in Figure 8 (a), the Random Forest F1 Score peaks for most dataset configurations when the window size is between 8 and 16. For the 50k dataset, a sharp dip in F1 score is observed at the smallest window size of 2, indicating that overly restricted windows might introduce local noise or capture insufficient context for stable z-score normalization.

Figure 8 (b) illustrates the Target CRs found across varying window sizes. While most dataset scales remain relatively stable within a tight range of 8 to 13 rules, the 20k dataset shows an upward trajectory, peaking at 20 rules when the window size reaches 32. This behaviour occurs because larger window sizes expand the historical context considered for each data point, allowing the algorithm to smooth out short-term fluctuations and unearth broader change rules that remain hidden within smaller, more localized windows.

Finally, Figure 8 (c) displays the Beta Runtime. The execution time for beta calculation remains highly stable and consistently low across all window sizes.

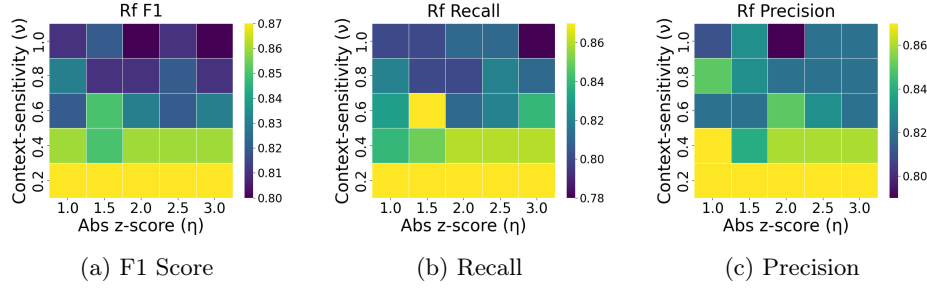


Fig. 9: Studying the effects of changing labelling thresholds.

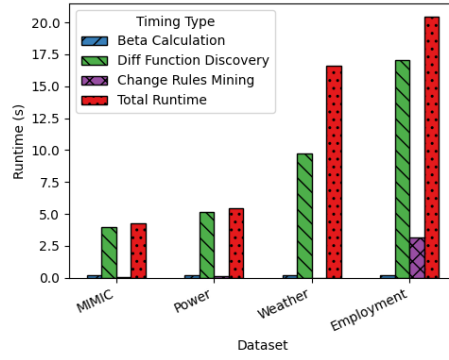


Fig. 10: Runtime for each module of the framework.

This confirms that varying the window size does not introduce severe computational overhead, making the system highly robust to parameter adjustments.

Exp-8: Varying absolute z-score threshold η and context-sensitivity threshold ν . Figure 9 (a)–(c) shows F1 score, recall, and precision as functions of the absolute z-score threshold η and context-sensitivity parameter ν . The highest F1 scores are concentrated along the bottom rows of the heatmap (low ν , 0.2–0.4) across all values of η , indicating that the random forest model performs best when abnormality labelling is driven primarily by the absolute z-score criterion rather than contextual deviation. Performance degrades progressively as ν increases, suggesting that over-weighting the context-relative condition introduces noise into the labelling process.

Exp-9: Ablation Study. Figure 10 illustrates the runtime breakdown of each major module of the CR-Miner framework across different datasets. Across all four datasets (Employment, MIMIC, Power, and Weather), differential function discovery consistently dominates the total execution time, accounting for the vast majority of the overhead. In contrast, (context-factor β) beta calculation takes a negligible fraction of the runtime across the board. The contribution of CR mining varies by dataset. It constitutes a significant portion of the execution

time for the Employee dataset compared to the other datasets. Consequently, the total runtime is primarily driven by the efficiency of the differential function discovery step, as expected.

7 Limitations

While CR-Miner demonstrates promising performance and expressiveness for discovering context-aware change rules, some limitations warrant discussion. The main limitation of our work is that the antecedent of Change Rules is restricted to one attribute. Change Rules are defined with respect to this ordering attribute, and all discovered rules are inherently conditioned on changes induced by it. Real-world datasets often exhibit multiple plausible orderings, and the current framework does not reason over these jointly. A natural extension to CR-Miner would be to support conjunctive antecedents across multiple ordering attributes. One promising direction is to construct a joint transition graph, such as a Dynamic Bayesian Network or a State-Space Model, which can capture the concurrent and interacting shifts of multiple ordering variables.

The window size $\mathcal{W}$ governs the local baseline used for z-normalization, and while CR-Miner is broadly robust to this choice for $\mathcal{W} \geq 8$, very small windows can introduce instability in the computed context factors.

8 Related Work

Exploring Change. Prior work on change exploration focuses on detecting and interpreting data evolution. Early approaches, such as [4], model the semantics of individual changes but do not capture interactions across attributes. The "change cube" framework [1] enables multi-dimensional analysis of temporal changes, but remains limited to exploration and does not model inter-attribute propagation effects. Clustering-based approaches for time-series data [2] identify common temporal patterns, but treat entity-attribute pairs independently, ignoring dependencies across attributes. Similarly, streaming-based frameworks [9] detect changes using statistical signals, but focus only on anomaly detection.

Data Dependencies. Data dependencies capture relationships between attributes. Order Dependencies (ODs) enforce ordering relationships but do not quantify attribute-level changes [19]. Sequential Dependencies (SDs) bound the changes in the consequent for ordered tuples [6]. However, it focuses only on changes in the consequent. Differential Dependencies (DDs) model bounded differences in both antecedent and consequent attributes across tuple pairs [18]. While expressive, they ignore tuple ordering and do not capture sequential patterns. Conditional Differential Dependencies (CDDs) extend DDs by restricting dependencies to subsets of tuples defined by condition sets [11]. However, CDDs also operate on unordered tuple pairs and do not capture the context of a change.

Statistical and ML Approaches. Statistical and machine learning methods detect abnormal changes by modeling expected behaviour and identifying deviations. Statistical techniques, such as Z-scores [7], model expected patterns of behaviour through assumptions about distributions. For temporal data, control

charts and statistical process control (SPC) are employed to identify outliers that exceed predefined control limits [14]. These methods are only effective when the data conforms to known distributions. Learning approaches such as isolation-based methods [12], deep autoencoder-based models [15], and sequence models like LSTMs [13] can capture complex patterns in data and detect deviations from learned behaviour. However, these methods focus on anomaly detection and do not provide interpretable relationships between attribute changes.

9 Conclusion and Future work

We introduce *change rules (CRs)*, which extend existing data quality rules by defining bounds on allowed changes in both the antecedent X and consequent Y attributes for tuples ordered on X . Our discovery framework identifies the relevant context for attribute changes and applies a scaling factor to adjust for out-of-norm changes. We introduce CR-Miner, a mining algorithm that generates and identifies minimal intervals leading to CRs satisfying a given support threshold. Our experimental evaluation shows that CR-Miner is scalable for increasing data sizes and number of attributes, with a 40-50% runtime improvement over existing differential dependency mining baselines. As next steps, we plan to study semantic extensions to consider conditional constraints where CRs hold. In addition, we will investigate scalability in higher-dimensional schemas where multiple candidate ordering attributes exist simultaneously.

References

1. Bleifuß, T., Bornemann, L., Johnson, T., Kalashnikov, D.V., Naumann, F., Srivastava, D.: Exploring change: A new dimension of data analytics. *Proceedings of the VLDB Endowment* **12**(2), 85–98 (2018)
2. Bornemann, L., Bleifuß, T., Kalashnikov, D., Naumann, F., Srivastava, D.: Data change exploration using time series clustering. *Datenbank-Spektrum* **18**, 79–87 (2018)
3. Celkan, T.T.: What does a hemogram say to us? *Turkish Archives of Pediatrics/Türk Pediatri Arşivi* **55**(2), 103 (2020)
4. Chawathe, S.S., Garcia-Molina, H.: Meaningful change detection in structured data. *ACM SIGMOD Record* **26**(2), 26–37 (1997)
5. Environment and Climate Change Canada: Daily climate observations (csv) dataset. Government of Canada, Meteorological Service of Canada Open Data (nd), <https://dd.weather.gc.ca/today/climate/observations/daily/csv/>, accessed: 2026-04-10
6. Golab, L., Karloff, H., Korn, F., Saha, A., Srivastava, D.: Sequential dependencies. *Proceedings of the VLDB Endowment* **2**(1), 574–585 (2009)
7. Heckert, N.A., Filliben, J.J., Croarkin, C.M., Hembree, B., Guthrie, W.F., Tobias, P., Prinz, J.: Handbook 151: Nist/sematech e-handbook of statistical methods (2002)
8. Johnson, A.E., Pollard, T.J., Shen, L., Lehman, L.w.H., Feng, M., Ghassemi, M., Moody, B., Szolovits, P., Anthony Celi, L., Mark, R.G.: Mimic-iii, a freely accessible critical care database. *Scientific data* **3**(1), 1–9 (2016)
9. Kanza, Y., Malik, R., Srivastava, D., Stone, C., Woodhull, G.: Data quality in data streams by modular change point detection (2023)

10. Kuang, S., Yang, H., Tan, Z., Ma, S.: Efficient differential dependency discovery. *Proceedings of the VLDB Endowment* **17**(7), 1552–1564 (2024)
11. Kwashie, S., Liu, J., Li, J., Ye, F.: Conditional differential dependencies (cdds). In: *East European Conference on Advances in Databases and Information Systems*. pp. 3–17. Springer (2015)
12. Liu, F.T., Ting, K.M., Zhou, Z.H.: Isolation forest. In: *2008 eighth IEEE international conference on data mining*. pp. 413–422. IEEE (2008)
13. Malhotra, P., Vig, L., Shroff, G., Agarwal, P., et al.: Long short term memory networks for anomaly detection in time series. In: *Proceedings*. vol. 89, p. 94 (2015)
14. Qiu, P.: Statistical process control charts as a tool for analyzing big data. *Big and Complex Data Analysis: Methodologies and Applications* pp. 123–138 (2017)
15. Sakurada, M., Yairi, T.: Anomaly detection using autoencoders with nonlinear dimensionality reduction. In: *Proceedings of the MLSDA 2014 2nd workshop on machine learning for sensory data analysis*. pp. 4–11 (2014)
16. Salam, A., El Hibaoui, A.: Power Consumption of Tetouan City. *UCI Machine Learning Repository* (2018), DOI: <https://doi.org/10.24432/C5B034>
17. Sharma, N.: Cr-miner: Change rules discovery repository (2026), https://github.com/nishtthasharma/Change_Rules
18. Song, S., Chen, L.: Differential dependencies: Reasoning and discovery. *ACM Transactions on Database Systems (TODS)* **36**(3), 1–41 (2011)
19. Szlichta, J., Godfrey, P., Golab, L., Kargar, M., Srivastava, D.: Effective and complete discovery of order dependencies via set-based axiomatization. *arXiv preprint arXiv:1608.06169* (2016)
20. United States Census Bureau: State naics detailed employment sizes dataset. *US Census Bureau Data* (2024), <https://www.census.gov/>, accessed: 2026-04-10